NetC Language Grammar

Complete Grammar for Recursive Descent Parser

```
Program → StatementList EOF

StatementList → Statement StatementList | ε

Statement →
    Declaration
    | Assignment
    | IfStmt
    | UntilStmt
    | IterateStmt
    | NetworkStmt
    | FeedStmt
    | ForwardStmt
    | ReturnStmt
    | LinkStmt
    | Comment
    | ε

LinkStmt → link StringLiteral ;

Declaration → DataType Identifier = Expr ;
    | DataType Identifier ;

DataType → text | dnum | cnum | flag

Assignment → Identifier = Expr ;

IfStmt → if ( Condition ) { StatementList }
    | if ( Condition ) { StatementList } else { StatementList }

UntilStmt → until ( Condition ) { StatementList }

IterateStmt → iterate ( Declaration ; Condition ; Assignment ) { StatementList }

NetworkStmt → network Identifier ( ParameterList ) { StatementList }
    | network Identifier ( ) { StatementList }

InitStmt → init ( ) { StatementList }

ParameterList → DataType Identifier
```

```
    | DataType Identifier , ParameterList

ReturnStmt → yield Expr ;

FeedStmt → feed Identifier ;

ForwardStmt → forward ( Expr ) ;

Condition → Expr RelOp Expr
    | Expr LogicalOp Expr
    | ! Condition
    | ( Condition )

Expr → Term
    | Term AddOp Expr

Term → Factor
    | Factor MulOp Term

Factor →
    IntegerLiteral
    | FloatLiteral
    | StringLiteral
    | BooleanLiteral
    | Identifier
    | Identifier ( ArgumentList )
    | ( Expr )
    | UnaryOp Factor

ArgumentList → Expr
    | Expr , ArgumentList
    | ε

AddOp → + | - | & | | | ^

MulOp → * | / | % | << | >>

UnaryOp → - | ! | ~ | ++ | --

RelOp → == | != | < | > | <= | >=

LogicalOp → && | ||

Comment → # any_characters_until_end_of_line
```